

JavaScript

What is JavaScript?

JavaScript (JS) is a programming language used to make web pages interactive. It works alongside **HTML** (structure) and **CSS** (design) to create dynamic web applications.

How to Run JavaScript?

Inside an HTML file (Recommended for beginners)

Html

1.

```
<html>
<body>
  <h1>Welcome to JavaScript</h1>
  <p id="Demo"></p>
  <script>
    document.getElementById("Demo").innerHTML = "Hello Word! ";
  </script>
</body>
</html>
```

2.

```
<html>
<body>
  <h1>Welcome to JavaScript</h1>
  <script>
    alert("Hello, World!");
  </script>
</body>
</html>
```

- This pops up an alert when you open the page in a browser.

In the Browser Console

- Open your browser (Chrome, Firefox, Edge)
- Right-click → Inspect → Go to the "Console" tab
- Type:

```
js
```

```
console.log("Hello, JavaScript!");
```

- Press **Enter** to see the output.

Using an External .js File

- Create a file called script.js
- Write:

```
js  
  
console.log("JavaScript is fun!");
```

- Link it in your HTML:

```
html  
  
<script src="script.js"></script>
```

JavaScript Comments

```
// This is a single-line comment
```

```
/*  
  This is a multi-line comment.  
  It can span multiple lines.  
*/
```

Variables

Variables store data values. In JavaScript, you can declare them using `var`, `let`, or `const`.

```
js  
  
let name = "Alice"; // String  
const age = 25;     // Number  
var isStudent = true; // Boolean
```

`let` → Block-scoped, can be reassigned.

`const` → Block-scoped, **cannot** be reassigned.

`var` → Function-scoped (older, avoid using it).

Ex-1

single `var` (Function-scoped, can be redeclared and reassigned)

js

```
var color = "Red";  
console.log(color); // Output: Red
```

```
var color = "Blue"; // ✓ Redeclaration allowed  
console.log(color); // Output: Blue
```

```
color = "Green"; // ✓ Reassignment allowed  
console.log(color); // Output: Green
```

Using `let` (Block-scoped, can be reassigned but not redeclared in the same block)

js

```
let score = 50;  
console.log(score); // Output: 50
```

```
score = 75; // ✓ Reassignment allowed  
console.log(score); // Output: 75
```

```
// let score = 100; ✗ Error: Cannot redeclare 'score' in the same block
```

Using `const` (Block-scoped, cannot be reassigned)

Js

```
const pi = 3.14;  
console.log(pi); // Output: 3.14
```

```
// pi = 3.1416; ✗ Error: Assignment to constant variable (cannot be reassigned)
```

Example with `let` inside a block

Js

```
let x = 10;  
if (true) {  
  let x = 20; // This is a different `x` inside this block  
  console.log("Inside block:", x); // Output: 20  
}  
console.log("Outside block:", x); // Output: 10
```

Example with `const` and objects (modifying properties but not reassigning the variable)

js

```
const person = { name: "Alice", age: 25 };  
console.log(person.name); // Output: Alice
```

```
person.age = 26; // ✔ Allowed (modifying property)
console.log(person.age); // Output: 26
```

```
// person = { name: "Bob", age: 30 }; ✖ Error: Cannot reassign a `const` object
```

JavaScript Operators

JavaScript Arithmetic Operators

JavaScript provides arithmetic operators to perform mathematical operations. Here are the basic ones with examples:

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	10 / 2	5
%	Modulus (Remainder)	10 % 3	1
**	Exponentiation	2 ** 3	8
++	Increment	let x = 5; x++	6
--	Decrement	let y = 5; y--	4

Examples in JavaScript Code:

js

```
let a = 10;
let b = 3;
```

```
console.log(a + b); // Addition: 13
console.log(a - b); // Subtraction: 7
console.log(a * b); // Multiplication: 30
console.log(a / b); // Division: 3.3333
console.log(a % b); // Modulus (Remainder): 1
console.log(2 ** 3); // Exponentiation: 8
```

```
// Increment and Decrement
```

```
let x = 5;
x++;
console.log(x); // Output: 6
```

```
let y = 5;
y--;
console.log(y); // Output: 4
```

Assignment Operators (Assign values to variables)

Operator Example Equivalent To

=	x = 10	x = 10
+=	x += 5	x = x + 5
-=	x -= 3	x = x - 3
*=	x *= 2	x = x * 2
/=	x /= 2	x = x / 2
%=	x %= 3	x = x % 3
**=	x **= 2	x = x ** 2

Example

js

```
let x = 10;  
x += 5; // x = x + 5 → x = 15  
console.log(x);
```

3. Comparison Operators (Compare values and return true or false)

Operator	Name	Example	Result
==	Equal to	5 == "5"	true
===	Strict equal (type + value)	5 === "5"	false
!=	Not equal	5 != 3	true
!==	Strict not equal	5 !== "5"	true
>	Greater than	10 > 5	true
<	Less than	10 < 5	false
>=	Greater than or equal to	5 >= 5	true
<=	Less than or equal to	5 <= 3	false

Example

js

```
console.log(10 > 5); // true  
console.log(10 == "10"); // true (loose comparison)  
console.log(10 === "10"); // false (strict comparison)  
console.log(5 !== "5"); // true
```

4. Logical Operators (Used in conditions)

Operator Name	Example	Result
&&	AND true && false	false
	OR	
!	NOT !true	false

Example

js

```
console.log(true && false); // false
console.log(true || false); // true
console.log(!true);        // false
```

5. Bitwise Operators (Operate at the binary level)

Operator	Name	Example
&	AND	5 & 1
	OR	
^	XOR	5 ^ 1
~	NOT	~5
<<	Left Shift	5 << 1
>>	Right Shift	5 >> 1

6. Ternary Operator (?:) (Shorter way to write if-else)

js

```
let age = 20;
let status = (age >= 18) ? "Adult" : "Minor";
console.log(status); // Output: Adult
```

7. Type Operators

Operator	Name	Example	Result
typeof	Returns the type of a variable	typeof "Hello"	"string"
instanceof	Checks if an object is an instance of a class obj instanceof Array		true/false

Example

js

```
console.log(typeof 42); // Output: "number"
console.log(typeof "Hello"); // Output: "string"
```

```
let arr = [1, 2, 3];
console.log(arr instanceof Array); // true
```

Summary

1. **Arithmetic Operators** (+, -, *, /, %, ++, --, **)
 2. **Assignment Operators** (=, +=, -=, *=, /=, %=, **=)
 3. **Comparison Operators** (==, ===, !=, !==, >, <, >=, <=)
 4. **Logical Operators** (&&, ||, !)
 5. **Bitwise Operators** (&, |, ^, ~, <<, >>)
 6. **Ternary Operator** (condition ? true : false)
 7. **Type Operators** (typeof, instanceof)
-

JavaScript Data Types

JavaScript has different data types that help store and manipulate values. These data types are categorized into **Primitive** and **Non-Primitive (Reference) Types**.

1. Primitive Data Types

Primitive types are immutable (cannot be changed) and store single values.

Data Type	Example	Description
String	"Hello"	Stores text
Number	42, 3.14	Stores integers and floating-point numbers
Boolean	true, false	Stores logical values
Undefined	let x;	Variable declared but not assigned a value
Null	let y = null;	Represents an empty or unknown value
BigInt	1234567890123456789n	Stores very large integers
Symbol	Symbol('id')	Unique identifier

Example of Primitive Data Types

js

```
let name = "Alice"; // String
let age = 25;       // Number
let isStudent = true; // Boolean
let height;         // Undefined
let emptyValue = null; // Null
let bigNumber = 9007199254740991n; // BigInt
let uniqueId = Symbol("id"); // Symbol
```

```
console.log(typeof name);    // "string"
console.log(typeof age);     // "number"
console.log(typeof isStudent); // "boolean"
console.log(typeof height);  // "undefined"
console.log(typeof emptyValue); // "object" (special case)
console.log(typeof bigNumber); // "bigint"
console.log(typeof uniqueId); // "symbol"
```

2. Non-Primitive (Reference) Data Types

These types store collections or complex structures.

Data Type	Example	Description
Object	{ name: "Alice", age: 25 }	Stores key-value pairs
Array	[1, 2, 3, 4]	Stores ordered lists
Function	function greet() {}	Defines a reusable block of code

Example of Non-Primitive Data Types

```
js
// Object (Key-Value Pairs)
let person = { name: "Alice", age: 25 };
console.log(person.name); // Output: Alice

// Array (List of Values)
let colors = ["Red", "Green", "Blue"];
console.log(colors[1]); // Output: Green

// Function (Reusable Code)
function greet() {
  return "Hello, World!";
}
console.log(greet()); // Output: Hello, World!
```

3. Special Cases

Dynamic Typing

JavaScript is **dynamically typed**, meaning a variable's type can change:

```
js
let value = 10;
console.log(typeof value); // "number"

value = "Hello";
console.log(typeof value); // "string"
```

Checking Data Types

Use typeof to check the type of a variable:

js

```
console.log(typeof 42);    // "number"
console.log(typeof "Hi");  // "string"
console.log(typeof true);  // "boolean"
console.log(typeof null);  // "object" (JavaScript bug)
console.log(typeof undefined); // "undefined"
console.log(typeof {});    // "object"
console.log(typeof []);    // "object" (arrays are objects)
console.log(typeof function(){}); // "function"
```

Summary

Primitive Types

✓ **String, Number, Boolean, Undefined, Null, BigInt, Symbol**

Non-Primitive (Reference) Types

✓ **Object, Array, Function**

JavaScript Function

Here's a simple function in JavaScript that adds two numbers:

javascript

// Function Definition

```
function addNumbers(a, b) {
    return a + b; // Return the sum of a and b
}
```

// Function Call

```
let sum = addNumbers(4, 5);
console.log(sum); // Output: 9
```

Explanation:

1. **function addNumbers(a, b):** This is the function declaration. The function is named addNumbers and accepts two parameters (a and b).
 2. **return a + b;:** The function returns the sum of a and b.
 3. **let sum = addNumbers(4, 5);:** This is calling the function addNumbers with the arguments 4 and 5, and it returns 9.
 4. **console.log(sum);:** This prints the result of the function call (9) to the console.
-

Another Example: A function to greet someone

javascript

```
function greet(name) {  
    return "Hello, " + name + "!";  
}
```

```
let greeting = greet("Alice");
```

```
console.log(greeting); // Output: Hello, Alice!
```

In this case, the function `greet` takes a `name` parameter and returns a greeting message.

Let me know if you need further details or more examples!

Objects

An **object** in JavaScript is created by grouping related data together in a single structure. The data inside an object can be of any type, including strings, numbers, arrays, other objects, and even functions.

Syntax of Creating an Object

javascript

```
let objectName = {  
    key1: value1,  
    key2: value2,  
    key3: value3  
};
```

- `objectName`: The name you assign to the object.
 - `key`: The property name (also called a key or field).
 - `value`: The value associated with that key (could be anything like a string, number, array, etc.).
-

Example of an Object

javascript

```
let person = {
  firstName: "John",
  lastName: "Doe",
  age: 30,
  greet: function() {
    return "Hello, " + this.firstName + " " + this.lastName;
  }
};
```

```
console.log(person.firstName); // Output: John
console.log(person.age);      // Output: 30
console.log(person.greet());  // Output: Hello, John Doe
```

More Complex Object Example

You can also have objects with nested objects, arrays, and even functions as values.

javascript

```
let car = {
  brand: "Toyota",
  model: "Corolla",
  year: 2020,
  features: ["Bluetooth", "Backup Camera", "Sunroof"],
  owner: {
    name: "Alice",
    age: 29
  },
  getCarInfo: function() {
    return `This is a ${this.year} ${this.brand} ${this.model}.`;
  }
};
```

```
console.log(car.getCarInfo());    // Output: This is a 2020 Toyota Corolla.  
console.log(car.features[1]);  
console.log(car.owner.name);    // Output: Alice
```

Creating a String

1. Using Single Quotes:

```
javascript  
let message = 'Hello, world!';
```

2. Using Double Quotes:

```
javascript  
let message = "Hello, world!";
```

3. Using Template Literals (Backticks):

Template literals allow for multi-line strings and string interpolation.

```
javascript  
let name = 'Alice';  
let greeting = `Hello, ${name}!`; // Interpolation  
console.log(greeting); // Output: Hello, Alice!
```

Common String Methods

JavaScript provides a lot of useful methods to manipulate strings. Here are some common ones:

1. **Length**: Get the length of the string.

```
javascript  
let text = "Hello";  
console.log(text.length); // Output: 5
```

2. **toUpperCase()**: Converts the string to uppercase.

```
javascript  
  
let message = "hello";  
console.log(message.toUpperCase()); // Output: HELLO
```

3. **toLowerCase()**: Converts the string to lowercase.

```
javascript
```

```
let message = "HELLO";  
console.log(message.toLowerCase()); // Output: hello
```

4. **charAt()**: Returns the character at a specific index.

```
javascript  
let message = "Hello";  
console.log(message.charAt(1)); // Output: e
```

5. **indexOf()**: Finds the index of the first occurrence of a substring.

```
javascript  
let message = "Hello, world!";  
console.log(message.indexOf("world")); // Output: 7
```

6. **slice()**: Extracts a section of the string and returns it as a new string.

```
javascript  
let message = "Hello, world!";  
console.log(message.slice(7, 12)); // Output: world
```

7. **replace()**: Replaces a substring with another string.

```
javascript  
let message = "Hello, world!";  
console.log(message.replace("world", "Alice")); // Output: Hello, Alice!
```

8. **split()**: Splits the string into an array of substrings based on a delimiter.

```
javascript  
let message = "Hello, world!";  
let words = message.split(", ");  
console.log(words); // Output: ['Hello', 'world!']
```

9. **trim()**: Removes whitespace from both ends of the string.

```
javascript  
let message = " Hello ";  
console.log(message.trim()); // Output: "Hello"
```

10. **includes()**: Checks if a substring exists within a string.

```
javascript  
Copy  
let message = "Hello, world!";  
console.log(message.includes("world")); // Output: true
```

Events in JavaScript

There are **two primary ways** to handle events:

1. **Inline Event Handling** (Not recommended for large applications):

You can specify the event handler directly in the HTML attribute (e.g., onclick).

html

```
<button onclick="alert('Button clicked!')">Click Me</button>
```

When the button is clicked, it triggers the alert message.

2. **Event Listeners** (Recommended method):

JavaScript provides the `addEventListener()` method to attach an event to an element and define the function to handle the event.

javascript

```
// Select the button element
```

```
let button = document.querySelector('button');
```

```
// Add an event listener
```

```
button.addEventListener('click', function() {
```

```
    alert('Button clicked!');
```

```
});
```

Array

1. **Using Square Brackets ([])**:

javascript

```
let fruits = ['apple', 'banana', 'cherry'];
```

```
console.log(fruits); // Output: ['apple', 'banana', 'cherry']
```

2. Using the Array() Constructor:

```
javascript
```

```
let numbers = new Array(1, 2, 3);
```

```
console.log(numbers); // Output: [1, 2, 3]
```

1.push() – Add Elements to the End of an Array

Adds one or more elements to the end of an array.

```
javascript
```

```
let fruits = ['apple', 'banana'];
```

```
fruits.push('cherry');
```

```
console.log(fruits); // Output: ['apple', 'banana', 'cherry']
```

2. pop() – Remove Elements from the End of an Array

Removes the last element from the array.

```
javascript
```

```
let fruits = ['apple', 'banana', 'cherry'];
```

```
let lastFruit = fruits.pop();
```

```
console.log(lastFruit); // Output: 'cherry'
```

```
console.log(fruits); // Output: ['apple', 'banana']
```

3. shift() – Remove Elements from the Beginning of an Array

Removes the first element from the array.

```
javascript
```

```
let fruits = ['apple', 'banana', 'cherry'];
```

```
let firstFruit = fruits.shift();
```

```
console.log(firstFruit); // Output: 'apple'
```

```
console.log(fruits); // Output: ['banana', 'cherry']
```

4. unshift() – Add Elements to the Beginning of an Array

Adds one or more elements to the beginning of the array.

```
javascript
```

```
let fruits = ['banana', 'cherry'];
```

```
fruits.unshift('apple');
```

```
console.log(fruits); // Output: ['apple', 'banana', 'cherry']
```

5. concat() – Merge Two or More Arrays

Combines two or more arrays and returns a new array.

```
javascript
```

```
let fruits = ['apple', 'banana'];
```

```
let moreFruits = ['cherry', 'date'];
```

```
let allFruits = fruits.concat(moreFruits);
```

```
console.log(allFruits); // Output: ['apple', 'banana', 'cherry', 'date']
```

6. join() – Join Array Elements into a String

Combines all elements of an array into a single string.

```
javascript
```

```
let fruits = ['apple', 'banana', 'cherry'];
```

```
let fruitsString = fruits.join(', ');
```

```
console.log(fruitsString); // Output: 'apple, banana, cherry'
```

7. slice() – Extract a Portion of an Array

Returns a shallow copy of a portion of the array, without modifying the original array.

```
javascript
```

```
let fruits = ['apple', 'banana', 'cherry', 'date'];
```

```
let selectedFruits = fruits.slice(1, 3);
```

```
console.log(selectedFruits); // Output: ['banana', 'cherry']
```

```
console.log(fruits); // Output: ['apple', 'banana', 'cherry', 'date']
```

8. splice() – Add/Remove Elements from Any Position

Adds or removes elements from a specific position in the array.

```
javascript
```

```
let fruits = ['apple', 'banana', 'cherry'];
```

```
fruits.splice(1, 1, 'blueberry', 'kiwi');
```

```
console.log(fruits); // Output: ['apple', 'blueberry', 'kiwi', 'cherry']
```

Date Methods

Once you have a Date object, there are various methods available to get or modify different components of the date and time.

1. Getting Date and Time Values

- **getFullYear():** Gets the full year (e.g., 2025).

javascript

```
let date = new Date();
```

```
console.log(date.getFullYear()); // Output: Current year
```

- **getMonth():** Gets the month (0-11). January is 0, February is 1, etc.

javascript

```
let date = new Date();
```

```
console.log(date.getMonth()); // Output: Current month (0 for January, 1 for February, etc.)
```

- **getDate():** Gets the day of the month (1-31).

javascript

```
let date = new Date();
```

```
console.log(date.getDate()); // Output: Current day of the month
```

- **getDay():** Gets the day of the week (0-6, where 0 is Sunday, 1 is Monday, etc.).

javascript

```
let date = new Date();
```

```
console.log(date.getDay()); // Output: Day of the week (0 for Sunday, 1 for Monday, etc.)
```

- **getHours():** Gets the hour (0-23).

javascript

```
let date = new Date();
```

```
console.log(date.getHours()); // Output: Current hour (24-hour format)
```

- **getMinutes():** Gets the minutes (0-59).

javascript

```
let date = new Date();
```

```
console.log(date.getMinutes()); // Output: Current minutes
```

- **getSeconds():** Gets the seconds (0-59).

javascript

```
let date = new Date();
```

```
console.log(date.getSeconds()); // Output: Current seconds
```

- **getMilliseconds():** Gets the milliseconds (0-999).

javascript

```
let date = new Date();  
console.log(date.getMilliseconds()); // Output: Current milliseconds
```

Math.random() – Generate a Random Number

Math.random() generates a pseudo-random floating-point number between 0 (inclusive) and 1 (exclusive). You can use this as the base to generate random numbers within a specified range.

javascript

```
let randomNumber = Math.random();  
console.log(randomNumber); // Output: A random number between 0 and 1
```

2. Rounding Random Numbers

To make the random number more useful, we typically round it to an integer or scale it to a specific range.

a) Rounding to the Nearest Integer using Math.round()

The Math.round() method rounds a number to the nearest integer.

javascript

```
let randomNumber = Math.random() * 10; // Random number between 0 and 10  
let roundedNumber = Math.round(randomNumber); // Round to the nearest integer  
console.log(roundedNumber);
```

b) Generating a Random Integer Within a Specific Range (Min, Max)

To generate a random integer within a specific range (inclusive of the minimum and maximum values), you can use Math.random(), Math.floor(), or Math.ceil() for rounding.

javascript

```
// Generate a random integer between 1 and 10 (inclusive)  
let min = 1;  
let max = 10;  
let randomInteger = Math.floor(Math.random() * (max - min + 1)) + min;  
console.log(randomInteger); // Output: A random integer between 1 and 10
```

If And Else Statements

The **if** statement evaluates a condition (expression). If the condition evaluates to true, the code block inside the if statement is executed. Otherwise, if an else block is provided, it is executed when the condition is false.

Syntax:

```
javascript
if (condition) {
    // Code to run if the condition is true
} else {
    // Code to run if the condition is false
}
```

Example:

```
javascript
let age = 20;

if (age >= 18) {
    console.log("You are an adult.");
} else {
    console.log("You are a minor.");
}
```

Output:

You are an adult.

Multiple Conditions with else if

You can chain multiple conditions using else if to check for more than one condition.

```
javascript
let temperature = 25;

if (temperature > 30) {
```

```
    console.log("It's too hot!");
} else if (temperature > 20) {
    console.log("It's a nice day.");
else if (temperature > 10) {
    console.log("It's a cool.");

} else {
    console.log("It's cold!");
}
```

Output:

rust

It's a nice day.

2. break Statement

The **break** statement is used to exit from a loop or a switch statement before its natural termination. It can also be used in for, while, or do-while loops to stop the loop early if a condition is met.

a) break in Loops

You can use break to exit a loop when a certain condition is met.

```
javascript
for (let i = 0; i < 10; i++) {
    if (i === 5) {
        console.log("Breaking the loop at i = 5");
        break; // Exit the loop when i is 5
    }
    console.log(i);
}
```

Output:

arduino

0
1
2
3

Breaking the loop at $i = 5$

For Loop

The for loop is used when you know in advance how many times you want to execute a statement or a block of code. It consists of three parts: initialization, condition, and iteration.

Syntax:

javascript

```
for (initialization; condition; iteration) {  
    // Code to be executed  
}
```

- **Initialization:** This step is executed once before the loop starts.
- **Condition:** The loop continues as long as this condition evaluates to true.
- **Iteration:** This is executed after every loop iteration and typically used to update the loop variable.

Example:

javascript

```
for (let i = 0; i < 5; i++) {  
    console.log(i);  
}
```

Output:

0
1
2
3
4

Here, i starts at 0, and the loop runs while i is less than 5. After each iteration, i is incremented by 1.

2. while Loop

The while loop is used when you want to repeat a block of code as long as a condition is true. The condition is checked before each iteration.

Syntax:

javascript

Copy

```
while (condition) {  
    // Code to be executed  
}
```

Example:

javascript

Copy

```
let i = 0;  
while (i < 5) {  
    console.log(i);  
    i++;  
}
```

Output:

Copy

0
1
2
3
4

Here, the condition $i < 5$ is checked before each iteration. As long as the condition is true, the loop will continue running.

3. do...while Loop

The do...while loop is similar to the while loop, but it guarantees that the code will run at least once, because the condition is checked **after** the loop executes.

Syntax:

javascript

Copy

```
do {  
    // Code to be executed  
} while (condition);
```

Example:

```
javascript
let i = 0;
do {
  console.log(i);
  i++;
} while (i < 5);
```

Output:

```
0
1
2
3
4
```

In this example, the loop body executes first, and then the condition $i < 5$ is checked. The loop will continue as long as the condition is true.

4. for...in Loop

The for...in loop is used to iterate over the **properties** of an object. It loops through the keys (property names) of an object or the indices of an array.

Syntax:

```
javascript
for (let key in object) {
  // Code to be executed
}
```

Example (for Object):

```
javascript
let person = { name: "Alice", age: 25, city: "New York" };
for (let key in person) {
  console.log(key + ": " + person[key]);
}
```

Output:

```
vbnet
```

name: Alice

age: 25

city: New York

Example (for Array):

javascript

```
let fruits = ['apple', 'banana', 'cherry'];  
for (let index in fruits) {  
  console.log(index + ": " + fruits[index]);  
}
```

Output:

makefile

0: apple

1: banana

2: cherry

5. for...of Loop

The for...of loop is used to iterate over the **values** of an iterable object (like arrays, strings, etc.). It's often used when you want to loop over the values of an array directly, rather than the indices.

Syntax:

javascript

```
for (let value of iterable) {  
  // Code to be executed  
}
```

Example (for Array):

javascript

```
let fruits = ['apple', 'banana', 'cherry'];  
for (let fruit of fruits) {  
  console.log(fruit);  
}
```

Output:

nginx

Copy

apple

banana

cherry

Set

You can create a Set using the Set constructor:

javascript

```
const set = new Set(); // Create an empty Set
const numbers = new Set([1, 2, 3, 4, 5]); // Create a Set with initial values
console.log(numbers); // Set(5) { 1, 2, 3, 4, 5 }
```

Set Methods

Here are the most common methods used with Set:

add(value)

Adds a value to the Set (if it's not already present).

javascript

```
const set = new Set();
set.add(1);
set.add(2);
set.add(2); // Duplicate values are ignored
console.log(set); // Set(2) { 1, 2 }
```

has(value)

Checks if a value exists in the Set.

javascript

```
console.log(set.has(1)); // true
console.log(set.has(3)); // false
```

delete(value)

Removes a value from the Set.

javascript

```
set.delete(2);  
console.log(set); // Set(1) { 1 }
```

clear()

Removes all elements from the Set.

javascript

```
set.clear();  
console.log(set.size); // 0
```

Map

You can create a Map using the Map constructor:

javascript

```
// Create an empty map  
const map = new Map();  
  
// Create a map with initial key-value pairs  
const user = new Map([  
  ['name', 'Alice'],  
  ['age', 25],  
  ['country', 'USA']  
]);  
  
console.log(user);  
// Output: Map(3) { 'name' => 'Alice', 'age' => 25, 'country' => 'USA' }
```

JSON

JSON stands for **JavaScript Object Notation**.

It's a lightweight data format used for storing and exchanging data, often between a server and a client.

- Based on JavaScript object syntax.
- Text-based format (easy to read and write).
- Supported by almost all programming languages.

JSON Syntax Rules

- ✓ Data is in **key-value pairs**.
- ✓ Keys must be **strings**.
- ✓ Values can be:

- Strings
- Numbers
- Booleans (true or false)
- Arrays
- Objects
- null

✓ Data is separated by **commas**.

✓ Curly braces {} hold **objects**.

✓ Square brackets [] hold **arrays**.

★ Example JSON Data

json

```
{
  "name": "Alice",
  "age": 25,
  "isEmployed": true,
  "skills": ["JavaScript", "React", "Node.js"],
  "address": {
    "street": "123 Main St",
    "city": "New York",
    "zip": "10001"
  }
}
```

💧 Working with JSON in JavaScript

Convert JSON to JavaScript Object (JSON.parse())

You can convert a JSON string into a JavaScript object using `JSON.parse()`:

javascript

```
const jsonData = '{"name": "Alice", "age": 25}';
const obj = JSON.parse(jsonData);
```

```
console.log(obj.name); // Alice
console.log(obj.age); // 25
```

Convert JavaScript Object to JSON (JSON.stringify())

You can convert a JavaScript object to a JSON string using `JSON.stringify()`:

javascript

```
const obj = {
  name: 'Bob',
  age: 30
};
```

```
const jsonString = JSON.stringify(obj);
```

```
console.log(jsonString);  
// Output: {"name":"Bob","age":30}
```

DOM

DOM stands for **Document Object Model**.

It's a **tree-like structure** that represents the structure of a webpage (**HTML document**) as **objects** that you can manipulate with JavaScript.

Accessing DOM Elements

You can access elements using **JavaScript** like this:

1. `getElementById()`

Finds an element by its id.

```
javascript  
  
const title = document.getElementById('title');  
console.log(title.textContent); // Hello, World!
```

2. `getElementsByClassName()`

Returns a **collection** of elements with the specified class.

```
javascript  
  
const messages = document.getElementsByClassName('message');  
console.log(messages[0].textContent); // This is a sample paragraph.
```

3. `getElementsByTagName()`

Returns a **collection** of elements with the specified tag name.

```
javascript  
  
const paragraphs = document.getElementsByTagName('p');  
console.log(paragraphs[0].textContent); // This is a sample paragraph.
```

4. `querySelector()`

Returns **the first matching element** using a CSS-style selector.

javascript

```
const title = document.querySelector('#title');  
console.log(title.textContent); // Hello, World!
```

5. querySelectorAll()

Returns **all matching elements** as a NodeList.

javascript

```
const messages = document.querySelectorAll('.message');  
messages.forEach(msg => console.log(msg.textContent));
```

Modifying DOM Elements

You can modify elements using JavaScript:

1. Change Text Content

javascript

```
const title = document.getElementById('title');  
title.textContent = 'New Title';
```

2. Change Inner HTML

You can insert **HTML** inside an element using `innerHTML`:

javascript

```
const message = document.querySelector('.message');  
message.innerHTML = '<strong>New content!</strong>';
```

3. Change Style

You can change styles using `style`:

javascript

```
title.style.color = 'red';  
title.style.fontSize = '30px';
```

4. Add or Remove Classes

Use `classList` to modify CSS classes:

javascript

```
title.classList.add('highlight');  
title.classList.remove('highlight');
```

```
title.classList.toggle('highlight'); // Adds if not present, removes if present
```

🔧 Creating and Removing Elements

1. Create New Element

You can create new elements using `createElement()`:

javascript

```
const newParagraph = document.createElement('p');
newParagraph.textContent = 'This is a new paragraph.';
document.body.appendChild(newParagraph);
```

2. Remove an Element

You can remove an element using `remove()`:

javascript

```
const title = document.getElementById('title');
title.remove();
```

3. Replace an Element

You can replace an element using `replaceChild()`:

javascript

```
const newTitle = document.createElement('h1');
newTitle.textContent = 'Welcome!';
document.body.replaceChild(newTitle, title);
```

🌟 Handling Events

You can add event listeners to handle user interactions:

1. addEventListener()

Add an event using `addEventListener()`:

javascript

```
const button = document.querySelector('button');
button.addEventListener('click', () => {
  alert('Button Clicked!');
});
```
